

ОС и БЛРЗ

1) Архитектура ОС (виртуальная память: виды и принцип работы)

Что такое архитектура ОС

Архитектура ОС — как ОС **организована внутри**: приложения работают через **системные вызовы** с ядром, ядро управляет памятью, процессами, I/O, драйверами.

Типовая цепочка:

app → **syscall** → **kernel** → **driver** → **device**

Виртуальная память: идея

Виртуальная память — механизм, при котором каждый процесс видит **своё виртуальное адресное пространство**, а ОС + железо (MMU) переводят виртуальные адреса в физические.

Зачем она нужна:

- **изоляция процессов** (процессы не лезут в память друг друга)
- **защита** (R/W/X права на страницы)
- **удобство** (каждый процесс “думает”, что память начинается с 0)
- можно использовать **swap** при нехватке RAM
- эффективно шарить библиотеки, mmap, кеш файлов

Принцип работы (коротко и понятно)

1. Процесс обращается к **виртуальному адресу (VA)**
2. **MMU** ищет перевод VA→PA:
 - сначала в **TLB** (кеш переводов)
 - если нет → идёт в **таблицы страниц (Page Table)**
3. Если страницы нет в RAM → **page fault**
4. ОС подгружает страницу (из файла/свапа), обновляет таблицы → работа продолжается

Страницы и кадры

- виртуальная память разбита на **страницы** (обычно 4KB)
- физическая RAM разбита на **кадры (frames)** такого же размера

Виды виртуальной памяти

Страничная (Paging) — основная сейчас

Сегментная (Segmentation) — логические куски (код/данные/стек), сейчас редко в “чистом виде”

Сегментно-страничная — комбинация

Важные термины (часто спрашивают)

- **TLB** — ускоряет перевод адресов
- **page fault** — обращение к странице, которой нет в RAM
- **swap** — выгрузка страниц на диск (медленно)
- **COW (copy-on-write)** — fork не копирует память сразу, копия делается при записи
- **mmap()** — отображение файла в память (файл как массив байт)

2) Мониторинг (точки для сбора информации)

Мониторинг — это сбор метрик о системе, чтобы понять:

- ✓ где узкое место (CPU / RAM / диск / сеть)
- ✓ что происходит с процессами
- ✓ почему тормозит/ошибка

Точки сбора информации (где можно посмотреть)

1) Уровень процесса (user space)

- CPU usage процесса
 - память процесса (RSS/VIRT)
 - число потоков
 - syscalls, ожидания
- Команды:
- `top`, `htop`
 - `ps aux`, `ps -eo pid,cmd,%cpu,%mem`
 - `pidstat -p <pid> 1`

2) Уровень ядра (kernel)

- планировщик (context switches)
- page faults (minor/major)
- работа VFS / файловых систем
- блочный ввод-вывод (I/O scheduler)

- прерывания (interrupts)

Команды/источники:

- `/proc` (про процессы и систему)
- `/proc/<pid>/status` , `/proc/meminfo` , `/proc/stat`
- `vmstat 1`
- `dmesg`

3) Точки в I/O (диск)

Важно отслеживать:

- latency (задержки)
- IOPS
- throughput (MB/s)
- очередь запросов

Команды

- `iostat -x 1`
- `iotop`
- `lsblk` , `blkid`

4) Точки в памяти

- free/used RAM
- cache/buffers
- swapping
- page faults

Команды:

- `free -h`
- `vmstat 1`
- `sar -r 1` (если есть sysstat)

5) Системные вызовы и трассировка

Когда надо понять “что программа делает”:

- `strace -p <pid>` — какие syscalls вызывает
- `lsof -p <pid>` — какие файлы/сокеты открыты

6) Продвинутый мониторинг: perf / eBPF / tracepoints

Используют, когда нужен “профиль” или ядро:

- `perf stat ...` — счётчики CPU
- `perf record/report` — профилирование
- eBPF (`bcc/bpfttrace`) — события ядра: `syscalls`, `block I/O`, `page cache hit/miss` и т.д.

Важно понимать разницу “CPU bound vs IO bound”

- **CPU bound** → процесс жрёт CPU, мало ожиданий
- **IO bound** → CPU низкий, но много “wait” (ожидание диска/сети), очередь, latency

3) Драйверы устройств

(major/minor, отличие от обычных программ, безопасность, сборка)

Что такое драйвер

Драйвер — код, который живёт в **ядре** и управляет конкретным устройством (диск, мышь, сеть).

Major/Minor номера (ls /dev)

Устройства в Linux представлены как “файлы” в `/dev`.

Пример:

```
ls -l /dev/sda # brw-rw---- 1 root disk 8, 0 ... /dev/sda
```

- первая буква:
 - `b` — block device (диск)
 - `c` — character device (мышь/клава/tty)
- **8, 0 = major, minor**
 - **major** — какой драйвер обслуживает класс устройств
 - **minor** — конкретное устройство/раздел

Полезно смотреть:

```
ls -l /dev | grep '^[^ld]'
```

(оставит устройства: `b` и `c`)

Чем драйвер отличается от обычной программы

Обычная программа:

- работает в user space
- если упала → упала только она

Драйвер:

- работает в kernel space

- ошибка → может уронить **всю систему** (kernel panic)
- нет обычных библиотек, ограничена стандартной средой

Ограничения/возможности драйверов

Возможности:

- прямой доступ к железу (регистры, шины)
- обработка прерываний
- DMA
- управление питанием
- создание интерфейса `/dev/...` для приложений

Ограничения:

- нельзя делать “что угодно” как в обычном приложении
- нельзя блокировать ядро надолго
- осторожно с памятью, конкурентностью и синхронизацией

Безопасность драйверов

Почему это важно:

- драйвер имеет высокий привилегированный доступ
- баг может стать уязвимостью

Механизмы защиты:

- права на `/dev/*` (кто может открыть устройство)
- принцип минимальных привилегий
- проверки входных данных в `ioctl`
- sandboxing частично через политики, namespaces (не всегда)

Сборка драйвера (kernel module)

Драйвер часто собирают как модуль `.ko`

Команды:

- `insmod module.ko` — вставить модуль
- `rmmod module` — удалить
- `lsmod` — список модулей
- `modinfo module.ko` — инфо

Сборка обычно через **Kbuild/Makefile** под текущий kernel headers.

4) Виртуализация и контейнеризация

Виртуализация (VM)

Виртуализация — запуск **полной ОС** внутри другой ОС с виртуальным железом.
Есть:

- **Type 1 (bare metal)** — гипервизор на железе (ESXi, Xen)
- **Type 2** — поверх ОС (VirtualBox)
В Linux популярно
- **KVM + QEMU**

Плюсы VM:

- ✓ сильная изоляция
- ✓ можно разные ОС
- ✓ хорошая безопасность между гостями

Минусы VM:

- ✗ тяжелее по ресурсам (ядро + ОС внутри)
- ✗ медленнее старт

Контейнеризация (Docker)

Контейнеры — это **не отдельная ОС**, а изолированные процессы одной ОС.

База:

- **Namespaces** (изоляция процессов, сети, mount, PID и т.д.)
- **cgroups** (лимиты CPU/RAM/IO)
- layered FS (overlay2)

Плюсы контейнеров:

- ✓ лёгкие и быстрые
- ✓ быстро стартуют
- ✓ удобно деплоить (образ = готовая среда)

Минусы:

- ✗ изоляция слабее, чем VM (общее ядро)
- ✗ если уязвимость ядра — риск больше

Самая важная разница (которую любят спрашивать)

- **VM** = виртуализируем железо → внутри своя ОС и ядро
- **Container** = изолируем процессы → ядро общее, “ОС” внутри нет как отдельной